

 Cronograma do Agente de Monitoramento

Equipe: Lucas Toterol Rodrigues & Caio Frederico Esquivel Lovera Arze

Objetivo: Desenvolver um agente nativo (Windows/Linux) que coleta métricas do sistema e logs de segurança, enviando-os via HTTP para o servidor central.

 Estrutura do Cronograma

Semana	Período	Foco Principal
1	21/04 – 27/04	Coleta local e definição do contrato
2	28/04 – 04/05	Primeira integração (envio HTTP)
3	05/05 – 11/05	Empacotamento e coleta avançada
4	12/05 – 18/05	Leitura de logs (auth.log)
5	19/05 – 25/05	Simulação de ataques e suporte à detecção
6	26/05 – 01/06	Coleta de rede (port scan) e preparação parcial
7	02/06 – 05/06	Estabilidade, instalação e documentação

◆ **Semana 1 | 21 – 27 abr**

 **Meta da Semana:** Ter o script Python capaz de ler métricas do sistema operacional localmente.

Tarefas

- **1.1** Criar repositório Git (se ainda não existir) e clonar localmente.
 - Criei o repositório chamado XXXXX
- **1.2** Configurar ambiente Python (recomendado: `venv`).
 - Ambiente foi configurado, venv se chama “ambiente virtual” e será usado python 3.12 (26/04/2026)
- **1.3** Escrever função para ler uso de CPU:
 - Linux: parse de `/proc/stat`
 - Windows: usar `psutil` (biblioteca externa)
- **1.4** Escrever função para ler uso de Memória RAM:
 - Linux: parse de `/proc/meminfo`
 - Windows: `psutil`
- **1.5** Executar script em loop com `time.sleep(10)` e imprimir valores no console.
- **1.6 Definir em conjunto com o Backend** o formato JSON do payload de métricas. Exemplo:

```
1 {  
2   "host_id": "client-01",  
3   "timestamp": "2026-04-21T14:35:00Z",  
4   "cpu_percent": 23.5,  
5   "memory_percent": 45.2  
6 }  
7
```

- **1.7** Testar script em **Linux** (VM ou máquina real) e em **Windows** (opcional, mas desejável).

Entregáveis da Semana

- Script funcional que coleta CPU e RAM e imprime no console.
- Formato JSON acordado com a equipe de Backend.

Semana 2 | 28 abr – 04 mai

 **Meta da Semana:** Enviar os dados coletados para o servidor Flask via HTTP POST.

Tarefas

- **2.1** Instalar biblioteca `requests` no ambiente do agente.
- **2.2** Substituir `print()` por `requests.post()` enviando o JSON para `http://<IP_DO_SERVIDOR>:5000/api/metrics` .
- **2.3** Configurar a URL do servidor via **variável de ambiente** ou arquivo de configuração (ex.: `AGENT_SERVER_URL`).
- **2.4** Implementar tratamento de erros de rede:

- Se a requisição falhar, aguardar 5 segundos e tentar novamente (evitar loop infinito de erros).
- **2.5** Adicionar cabeçalho `Content-Type: application/json`.
- **2.6 Testar integração real:**
 - Subir o backend Flask (rodando em container Docker) em uma máquina da rede.
 - Executar o script do agente e verificar nos logs do Flask se os dados estão chegando.
- **2.7** Ajustar o payload JSON conforme feedback do backend.

✓ Entregáveis da Semana

- Agente enviando métricas reais para o servidor.
- Código com tratamento básico de falhas de conexão.

♦ Semana 3 | 05 – 11 mai

🎯 **Meta da Semana:** Empacotar o agente como executável e adicionar coleta de Disco e Rede.

📋 Tarefas

- **3.1** Adicionar coleta de uso de Disco:
 - Linux: `df -h` ou `os.statvfs()`
 - Windows: `psutil.disk_usage('C:\\')`
- **3.2** Adicionar coleta de tráfego de Rede:
 - Bytes enviados/recebidos (`/proc/net/dev` no Linux; `psutil.net_io_counters()` no Windows).
- **3.3** Atualizar payload JSON para incluir `disk_percent` e `network_bytes_sent` / `network_bytes_recv`.
- **3.4 Empacotar com PyInstaller:**
 - Gerar executável único para Linux:


```
pyinstaller --onefile --name linux-agent agent.py
```
 - Gerar executável único para Windows (se possível):


```
pyinstaller --onefile --name windows-agent.exe agent.py
```
- **3.5** Testar o executável gerado em uma VM Linux limpa (sem Python instalado).
- **3.6** Criar uma estrutura de diretórios padronizada:
 - Linux: `/opt/monitor-agent/linux-agent`
 - Windows: `C:\Program Files\MonitorAgent\windows-agent.exe`

✓ Entregáveis da Semana

- Executável funcional (Linux) que coleta CPU, RAM, Disco e Rede.
 - Payload JSON atualizado e validado com o Backend.
-

♦ Semana 4 | 12 – 18 mai

🎯 **Meta da Semana:** Ler logs do sistema (`/var/log/auth.log`) e enviá-los ao servidor.

📋 Tarefas

- **4.1** Estudar como o agente terá permissão para ler `/var/log/auth.log` (necessário `sudo` ou adicionar usuário ao grupo `adm`).
- **4.2** Implementar leitura incremental do arquivo de log:
 - Guardar a última posição lida (em bytes) em um arquivo de estado (ex.: `authlog.pos`).
 - A cada ciclo, ler apenas as novas linhas adicionadas.
- **4.3** Filtrar apenas linhas relevantes (ex.: contendo `"Failed password"` ou `"Accepted password"`).
- **4.4** Definir formato JSON para envio de logs com o Backend. Exemplo:

```
1 {  
2   "host_id": "client-01",  
3   "timestamp": "2026-04-21T14:35:00Z",  
4   "log_type": "auth",  
5   "raw_line": "Apr 21 14:35:00 server sshd[123]: Failed password for root from 192.168.1.100"  
6 }  
7
```

- **4.5** Enviar logs para `POST /api/logs` (endpoint que o Backend deve disponibilizar).
- **4.6** Testar envio contínuo de logs junto com métricas (multithreading ou loop único alternado).

✓ Entregáveis da Semana

- Agente lendo `auth.log` de forma eficiente e enviando logs ao servidor.
 - Arquivo de estado funcionando (não reenvia logs antigos após reinicialização).
-

♦ Semana 5 | 19 – 25 mai

🎯 **Meta da Semana:** Suporte à detecção de Brute Force e testes de simulação.

📋 Tarefas

- **5.1** Garantir que o agente está enviando logs de falha de autenticação com todos os campos necessários (IP de origem, usuário, timestamp).

- **5.2 Criar script auxiliar de simulação de ataque** (para testes internos):
 - Pode ser um script Bash que executa `ssh invalid@localhost` várias vezes ou usa `hydra`.
 - **Importante:** Executar apenas em ambiente controlado (VM de testes).
- **5.3** Verificar se o backend consegue gerar alertas a partir dos logs enviados.
- **5.4** Ajustar a frequência de envio de logs se necessário (evitar flood na rede).
- **5.5** Adicionar suporte a logs do Windows (Event Log de Segurança) **se houver tempo**. Caso contrário, documentar como limitação.

✓ Entregáveis da Semana

- Agente enviando logs que permitem ao Backend detectar brute force.
- Script de simulação pronto para uso nas demonstrações.

♦ Semana 6 | 26 mai – 01 jun

🎯 **Meta da Semana:** Coleta de conexões de rede (para Port Scan) e preparação para apresentação parcial (29/05).

📋 Tarefas

- **6.1** Implementar coleta de conexões de rede ativas:
 - Linux: executar `ss -tn` ou ler `/proc/net/tcp`.
 - Windows: `netstat -an` (via subprocess) ou `psutil.net_connections()`.
- **6.2** Filtrar conexões do tipo `SYN_SENT` ou `ESTABLISHED` que possam indicar varredura.
- **6.3** Definir com o Backend um payload para eventos de rede. Exemplo:

```

1 {
2   "host_id": "client-01",
3   "timestamp": "...",
4   "connections": [
5     {"local_port": 22, "remote_ip": "192.168.1.100", "state": "ESTABLISHED"}
6   ]
7 }
8
```

- **6.4** Enviar esses dados periodicamente ou quando um limiar for atingido (ex.: > 10 conexões SYN para portas diferentes).
- **6.5 Preparar demonstração para o professor (29/05):**
 - Garantir que o agente está estável e enviando dados.
 - Ter uma VM Linux com o agente instalado como serviço (próxima semana) para mostrar.
 - Alinhar com o time de Frontend para que os dados apareçam no dashboard.

✓ Entregáveis da Semana

- Agente coletando informações de rede.
 - Agente pronto para ser apresentado na parcial.
-

♦ Semana 7 | 02 – 05 jun

🎯 **Meta da Semana:** Transformar o agente em **serviço do sistema** e documentar instalação.

📋 Tarefas

• 7.1 Linux: Criar arquivo de serviço systemd

- Criar `/etc/systemd/system/linux-agent.service` com:

```
1 [Unit]
2 Description=Monitor Agent
3 After=network.target
4
5 [Service]
6 ExecStart=/opt/monitor-agent/linux-agent
7 Restart=always
8 User=root (ou usuário com permissão para ler logs)
9
10 [Install]
11 WantedBy=multi-user.target
12
```

- Habilitar e iniciar: `systemctl enable linux-agent && systemctl start linux-agent`

• 7.2 Testar se o agente inicia automaticamente após reboot.

• 7.3 Windows: Configurar como serviço usando NSSM

- Baixar NSSM e executar: `nssm install MonitorAgent`
- Apontar para o executável e definir início automático.

• 7.4 Criar script de instalação (`install.sh` e `install.bat`) que:

- Copia o executável para o diretório correto.
- Cria o serviço (Linux) ou entrada no registro (Windows).

• 7.5 Escrever **README.md** específico do agente contendo:

- Como instalar.
- Como configurar a URL do servidor.
- Como verificar logs do agente.

• 7.6 Teste de estabilidade: deixar o agente rodando por 24h e verificar se há vazamento de memória ou crashes.

✓ Entregáveis da Semana

- Agente instalável como serviço em Linux (e opcionalmente Windows).
 - Documentação clara para o usuário final.
-

🎤 Período de Pitch (06 – 12 jun)

Tarefas do Agente para o Pitch

- **P.1** Garantir que o agente está funcionando na VM que será usada na apresentação.
 - **P.2** Preparar um "roteiro de demonstração":
 - a. Mostrar agente rodando (`systemctl status linux-agent`).
 - b. Simular ataque SSH.
 - c. Mostrar o alerta aparecendo no dashboard.
 - **P.3** Estar disponível para suporte técnico durante a apresentação (conexão, firewall, etc.).
-

⚠ Observações Importantes

1. Permissões de Leitura de Logs:

No Linux, o arquivo `/var/log/auth.log` pertence ao grupo `adm`. Para evitar rodar o agente como `root`, adicione o usuário que executa o agente ao grupo `adm`:

```
sudo usermod -a -G adm <seu_usuario>
```

2. Firewall:

Certifiquem-se de que a porta do servidor Flask (ex.: `5000`) está acessível a partir da máquina onde o agente roda.

3. Cross-Platform:

Priorizem o Linux, pois é o foco do projeto. O suporte ao Windows é um diferencial, mas não pode atrasar o core.

4. Sincronização com Backend:

Antes de codificar qualquer payload, **validem com Douglas e Fernando** o formato exato esperado. Mudanças de última hora quebram a integração